Examination Result and Grace-Mark Validation:

Process marks across multiple subjects, apply subject-wise pass criteria, and a specified grace-mark policy, and generate final classifications without exceeding maximum marks.

Problem Statement

Develop a program to process marks across multiple subjects. The program must check the minimum passing marks for each subject and determine whether the student has passed or failed.

If the student fails a subject by a small number of marks, the program should apply a grace-mark policy, provided the student meets the eligibility conditions. Grace marks must not cause the final marks to exceed the maximum marks for any subject. Finally, the program should calculate the total marks, percentage, and final classification, and display the original marks, grace marks awarded, and final marks for each subject.

Python Code

```python
subjects = 5
pass_marks = 40
grace_limit = 5
max_marks = 100

original_marks = []
grace_marks = [0] * subjects

# Input marks for five subjects
for i in range(subjects):
    mark = int(input(f"Enter marks for Subject {i + 1}: "))
    original_marks.append(mark)

# Validate marks
if any(mark < 0 or mark > max_marks for mark in original_marks):
    print("\nInvalid marks! Marks must be between 0 and 100.")
else:
    final_marks = original_marks.copy()

    failed_subjects = [i for i in range(subjects)  if original_marks[i] < pass_marks]

    # Apply grace marks if exactly one subject failed and has 35-39 marks
    if len(failed_subjects) == 1:
        i = failed_subjects[0]
        if 35 <= original_marks[i] <= 39:
            required_grace = pass_marks - original_marks[i]
            if required_grace <= grace_limit:
                grace_marks[i] = required_grace
                final_marks[i] = min(original_marks[i] + required_grace, max_marks)

    total = sum(final_marks)
    percentage = (total / (subjects * max_marks)) * 100

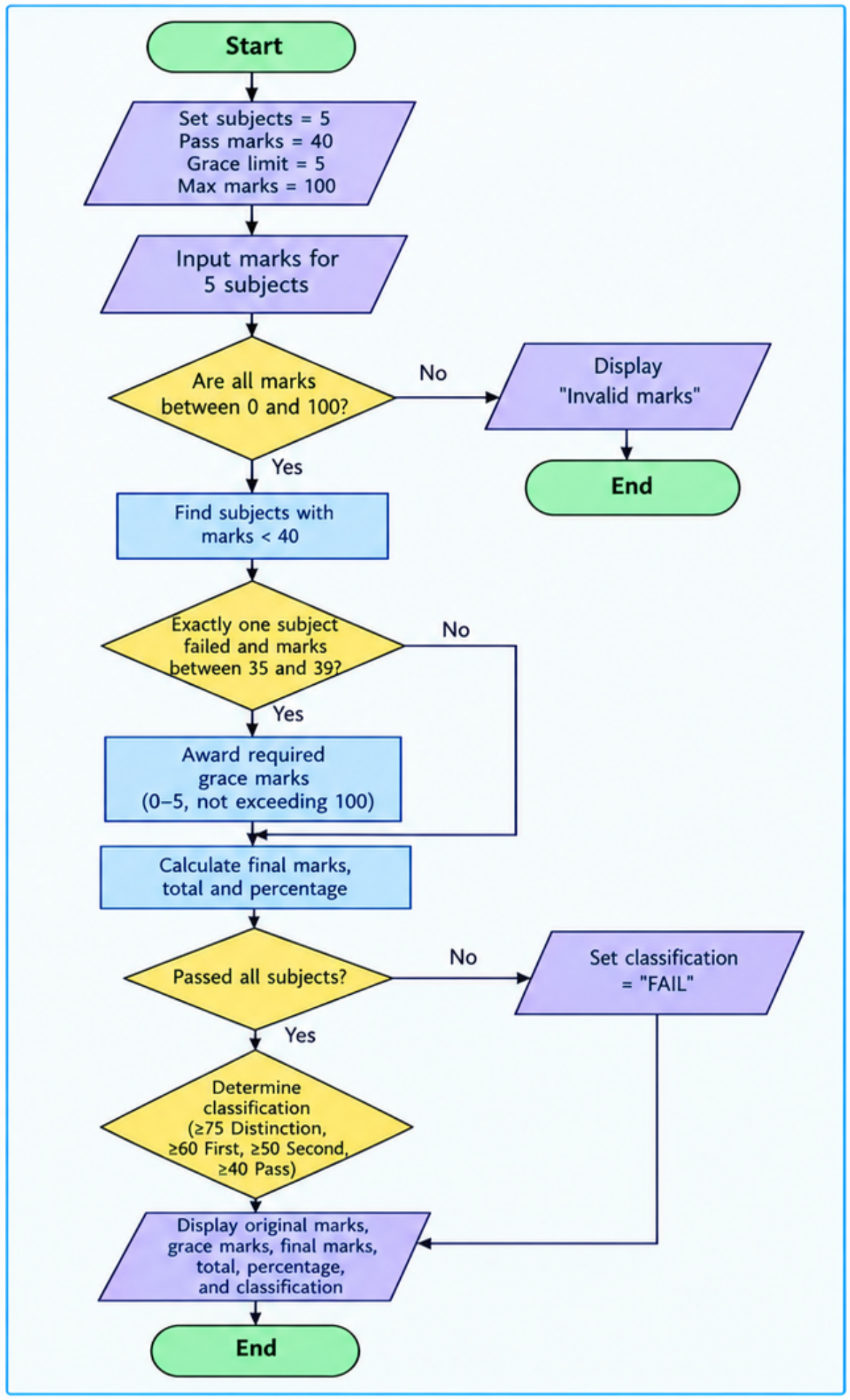
    if any(mark < pass_marks for mark in final_marks):
        classification = "FAIL"
    elif percentage >= 75:
        classification = "DISTINCTION"
    elif percentage >= 60:
        classification = "FIRST CLASS"
    elif percentage >= 50:
        classification = "SECOND CLASS"
    else:
        classification = "PASS"

    # Display result
    print("\n========== EXAMINATION RESULT ==========")
    print(f"Subject\tOriginal\tGrace\tFinal")
    for i in range(subjects):
        print(f"{i + 1}\t{original_marks[i]}\t{grace_marks[i]}\t{final_marks[i]}")
    print("-" * 45)
    print(f"Total Marks: {total} / {subjects * max_marks}")
    print(f"Percentage: {percentage:.2f}%")
    print("Classification:", classification)
```

Program Output

```
Enter marks for Subject 1: 78
Enter marks for Subject 2: 65
Enter marks for Subject 3: 38
Enter marks for Subject 4: 82
Enter marks for Subject 5: 71

========== EXAMINATION RESULT ==========
Subject        Original        Grace          Final
    1             78             0             78
    2             65             0             65
    3             38             2             40
    4             82             0             82
    5             71             0             71
---------------------------------------------
Total Marks: 336 / 500
Percentage: 67.20%
Classification: FIRST CLASS
```



2. Attendance Recovery Computation

Calculate attendance percentage and the minimum consecutive classes needed to meet an eligibility threshold.

Problem Statement

Given classes attended and total classes held, compute current attendance. For each future scenario, determine the smallest number of consecutive classes the student must attend to reach the required percentage. Validate inputs and report when recovery is impossible under the stated scenario.

Python Code

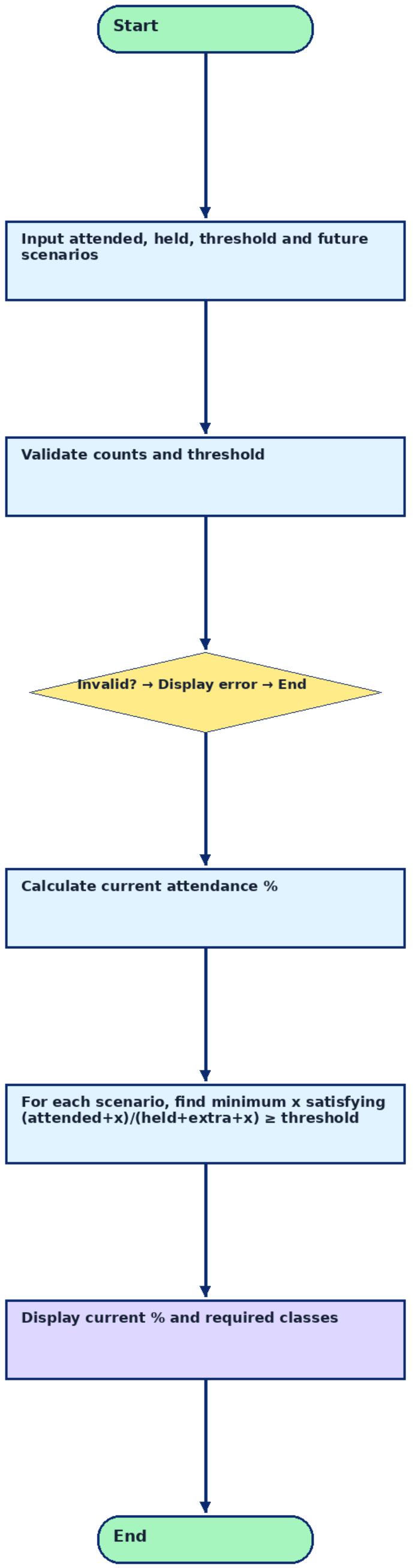
```
attended = int(input("Classes attended: "))
held = int(input("Classes held: "))
threshold = float(input("Eligibility percentage: "))
future_scenarios = [0, 5, 10]  # extra classes held and attended

if held <= 0 or attended < 0 or attended > held:
    print("Invalid attendance data")
elif not 0 < threshold <= 100:
    print("Invalid threshold")
else:
    current = attended / held * 100
    print(f"Current attendance: {current:.2f}%")
    for extra_held in future_scenarios:
        # x future classes are all attended
        x = 0
        while (attended + x) / (held + extra_held + x) * 100 < threshold:
            x += 1
            if x > 100000:
                break
        print(f"Scenario +{extra_held} held: attend {x} more")
```

Sample Program Output

```
Classes attended: 36
Classes held: 50
Eligibility percentage: 75
Current attendance: 72.00%
Scenario +0 held: attend 6 more
Scenario +5 held: attend 21 more
Scenario +10 held: attend 36 more
```

Flowchart / Procedure



3. ATM Transaction Sequence Simulation

Process deposits and withdrawals while enforcing balance and transaction rules.

Problem Statement

Simulate a sequence of transactions starting from an opening balance. Accept valid deposits, reject non-positive amounts, reject withdrawals exceeding the balance or per-transaction limit, update the balance only for accepted transactions, and display a transaction summary.

Python Code

```
balance = float(input("Opening balance: "))
limit = float(input("Withdrawal limit: "))
transactions = [
    ("deposit", 500),
    ("withdraw", 200),
    ("withdraw", 900),
    ("deposit", -50)
]
accepted = rejected = 0

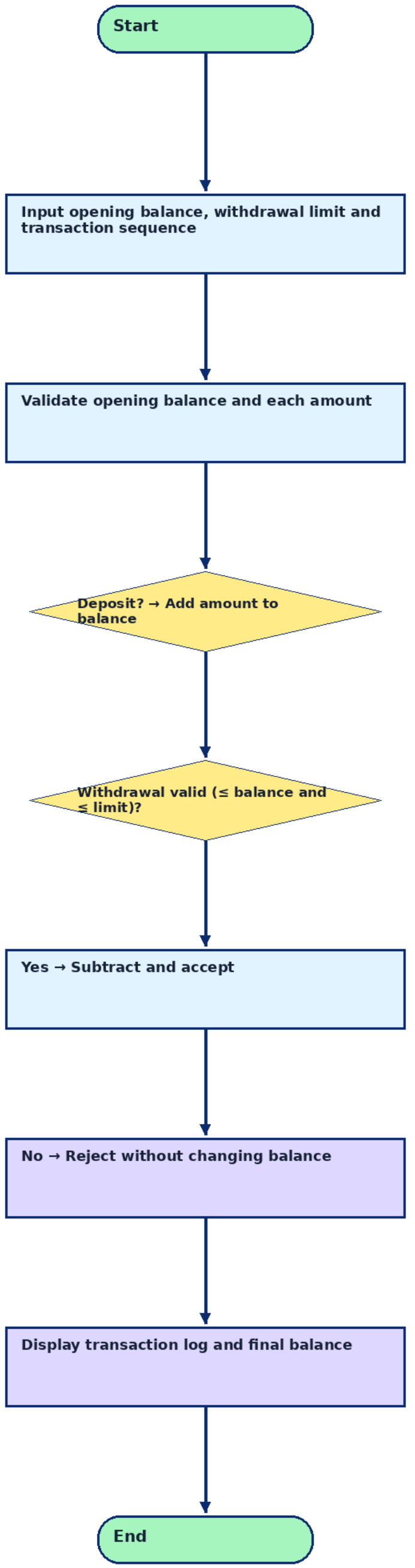
for kind, amount in transactions:
    if amount <= 0:
        print("Rejected:", kind, amount, "- invalid amount")
        rejected += 1
    elif kind == "deposit":
        balance += amount
        accepted += 1
        print("Deposit accepted:", amount)
    elif kind == "withdraw" and amount <= limit and amount <= balance:
        balance -= amount
        accepted += 1
        print("Withdrawal accepted:", amount)
    else:
        print("Rejected withdrawal:", amount)
        rejected += 1

print(f"Final balance: {balance:.2f}")
print("Accepted:", accepted, "Rejected:", rejected)
```

Sample Program Output

```
Opening balance: 1000.00
Withdrawal limit: 800.00
Deposit accepted: 500
Withdrawal accepted: 200
Rejected withdrawal: 900
Rejected: deposit -50 - invalid amount
Final balance: 1300.00
Accepted: 2 Rejected: 2
```

Flowchart / Procedure



4. Warehouse Inventory Reconciliation

Compare expected and actual stock and calculate replenishment needs for multiple products.

Problem Statement

For each product, reconcile opening stock plus incoming units minus dispatched units against the recorded closing stock. Flag discrepancies, prevent negative quantities, and calculate replenishment needed to reach a target stock level.

Python Code

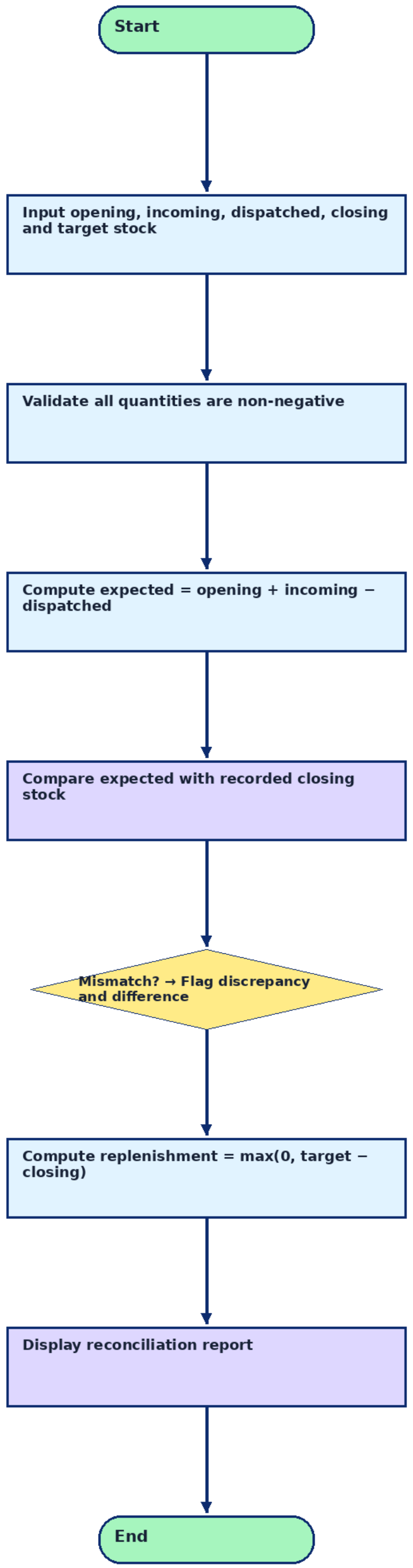
```
products = [
    {"name": "Bolts", "open": 100, "in": 40, "out": 90,
     "closing": 45, "target": 80},
    {"name": "Nuts", "open": 60, "in": 10, "out": 25,
     "closing": 40, "target": 50}
]

for p in products:
    expected = p["open"] + p["in"] - p["out"]
    discrepancy = p["closing"] - expected
    replenish = max(0, p["target"] - p["closing"])
    status = "MATCH" if discrepancy == 0 else "DISCREPANCY"
    print(p["name"], status,
          "expected=", expected,
          "recorded=", p["closing"],
          "difference=", discrepancy,
          "replenish=", replenish)
```

Sample Program Output

```
Bolts DISCREPANCY expected= 50 recorded= 45
difference= -5 replenish= 35
Nuts DISCREPANCY expected= 45 recorded= 40
difference= -5 replenish= 10
```

Flowchart / Procedure



5. Smart Parking Fee Management

Compute parking charges from timestamps, vehicle type, daily caps and penalties.

Problem Statement

Validate entry and exit times, calculate parked duration including overnight stays, apply vehicle-specific hourly rates and daily maximums, and add any applicable penalty. Reject invalid or reversed timestamps and show the billing breakdown.

Python Code

```
from datetime import datetime, timedelta
from math import ceil

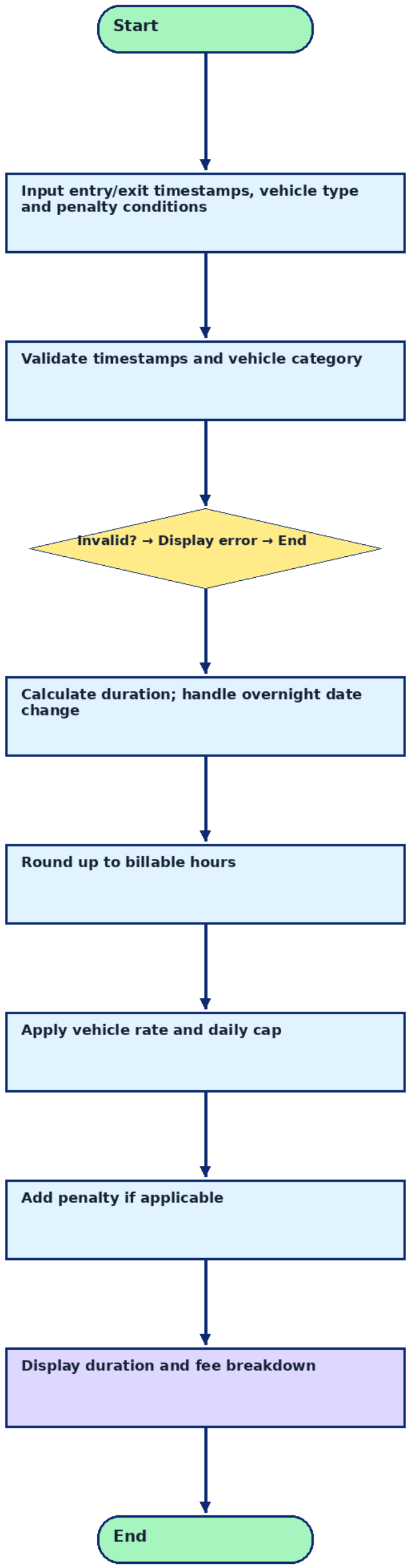
entry = datetime.strptime("2026-10-23 22:30", "%Y-%m-%d %H:%M")
exit_time = datetime.strptime("2026-10-24 02:10", "%Y-%m-%d %H:%M")
vehicle = "car"
rates = {"bike": 20, "car": 40, "van": 60}
daily_caps = {"bike": 100, "car": 200, "van": 300}
penalty = 0

if exit_time < entry or vehicle not in rates:
    print("Invalid parking input")
else:
    minutes = int((exit_time - entry).total_seconds() / 60)
    hours = ceil(minutes / 60)
    raw_fee = hours * rates[vehicle]
    days = max(1, ceil(minutes / 1440))
    fee = min(raw_fee, daily_caps[vehicle] * days) + penalty
    print("Duration:", hours, "billable hours")
    print("Base fee:", min(raw_fee, daily_caps[vehicle] * days))
    print("Penalty:", penalty)
    print("Total fee:", fee)
```

Sample Program Output

```
Duration: 4 billable hours
Base fee: 160
Penalty: 0
Total fee: 160
Example uses car rate = 40 per hour.
```

Flowchart / Procedure



6. Multi-Day Water Tank Simulation

Track tank levels over time and detect overflow, shortage and threshold crossings.

Problem Statement

Starting with an initial water level, process each interval's inflow and outflow. Report overflow above tank capacity, shortage below zero, clamp the physical level to the valid range, and identify the first interval when the level reaches a target threshold.

Python Code

```
capacity = 1000
level = 600
threshold = 850
flows = [(200, 100), (100, 250), (300, 50), (50, 80)]
first_reached = None

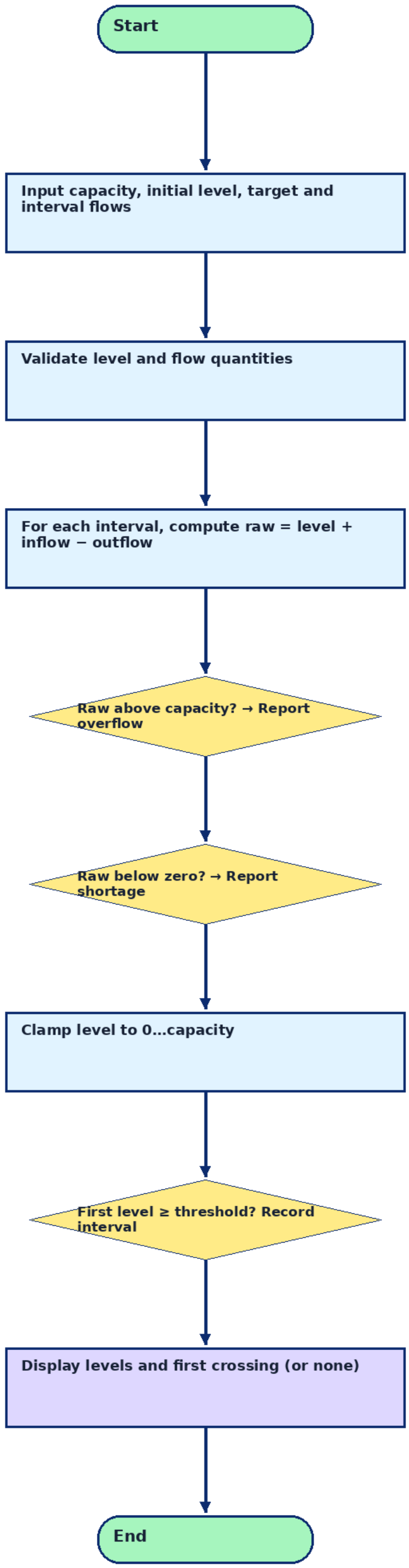
for day, (inflow, outflow) in enumerate(flows, start=1):
    raw = level + inflow - outflow
    if raw > capacity:
        print(f"Day {day}: overflow {raw - capacity}")
    if raw < 0:
        print(f"Day {day}: shortage {-raw}")
    level = min(capacity, max(0, raw))
    print(f"Day {day}: level={level}")
    if first_reached is None and level >= threshold:
        first_reached = day

print("First threshold interval:", first_reached)
```

Sample Program Output

```
Day 1: level=700
Day 2: level=550
Day 3: level=800
Day 4: level=770
First threshold interval: None
```

Flowchart / Procedure



7. Student Performance Trend Analysis

Compare exam scores, identify changes and weak subjects, and classify performance.

Problem Statement

Read scores for several subjects across multiple examinations. Calculate score changes between exams, identify improvement or decline, mark subjects that remain below a defined threshold in every exam, and assign a classification using the overall average.

Python Code

```
scores = {
    "Math": [58, 66, 72],
    "Science": [70, 65, 62],
    "English": [45, 48, 49]
}
weak_limit = 50

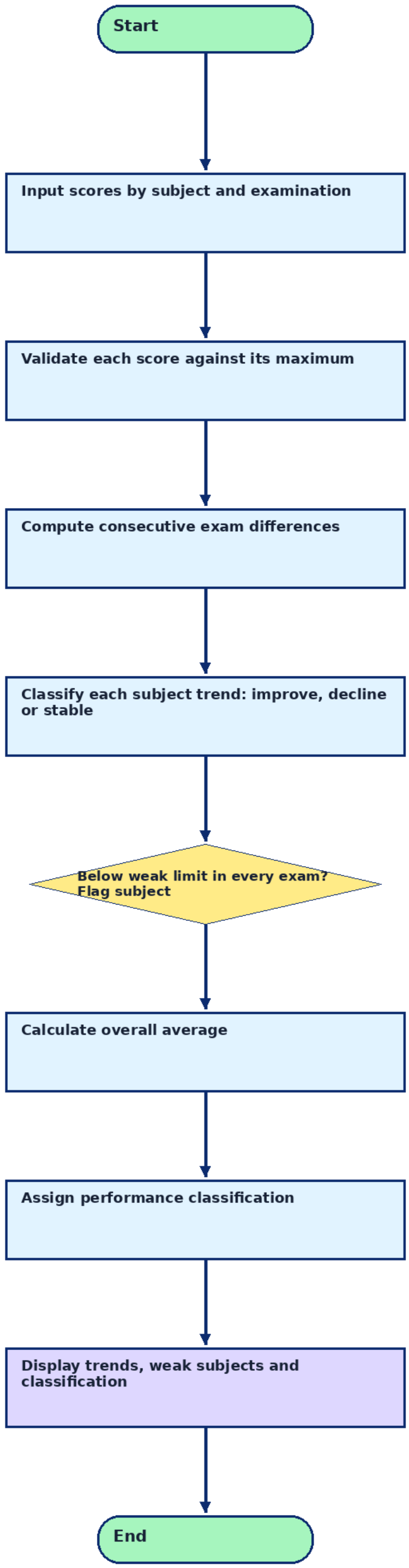
for subject, marks in scores.items():
    changes = [marks[i] - marks[i-1]
               for i in range(1, len(marks))]
    weak = all(m < weak_limit for m in marks)
    trend = "improving" if changes and changes[-1] > 0 else (
        "declining" if changes and changes[-1] < 0 else "stable")
    print(subject, "changes=", changes,
          "trend=", trend, "consistently weak=", weak)

all_marks = [m for values in scores.values() for m in values]
avg = sum(all_marks) / len(all_marks)
grade = "Excellent" if avg >= 75 else (
    "Good" if avg >= 60 else "Needs Improvement")
print(f"Average: {avg:.2f}; Classification: {grade}")
```

Sample Program Output

```
Math changes= [8, 6] trend= improving consistently weak= False
Science changes= [-5, -3] trend= declining consistently weak= False
English changes= [3, 1] trend= improving consistently weak= True
Average: 59.44; Classification: Needs Improvement
```

Flowchart / Procedure



8. Fuel Consumption and Route Comparison

Estimate fuel, cost and reserve feasibility for multiple vehicle choices.

Problem Statement

For each vehicle and route option, calculate trip fuel from distance and mileage, add reserve fuel, compute journey cost using fuel price, and check whether the vehicle's available fuel or budget is sufficient. Select the lowest-cost feasible option.

Python Code

```
distance_km = 320
fuel_price = 105
reserve_pct = 10
budget = 3000
vehicles = {
    "Car A": {"mileage": 16, "fuel_available": 25},
    "Car B": {"mileage": 20, "fuel_available": 20},
    "Van":   {"mileage": 10, "fuel_available": 40}
}
feasible = []

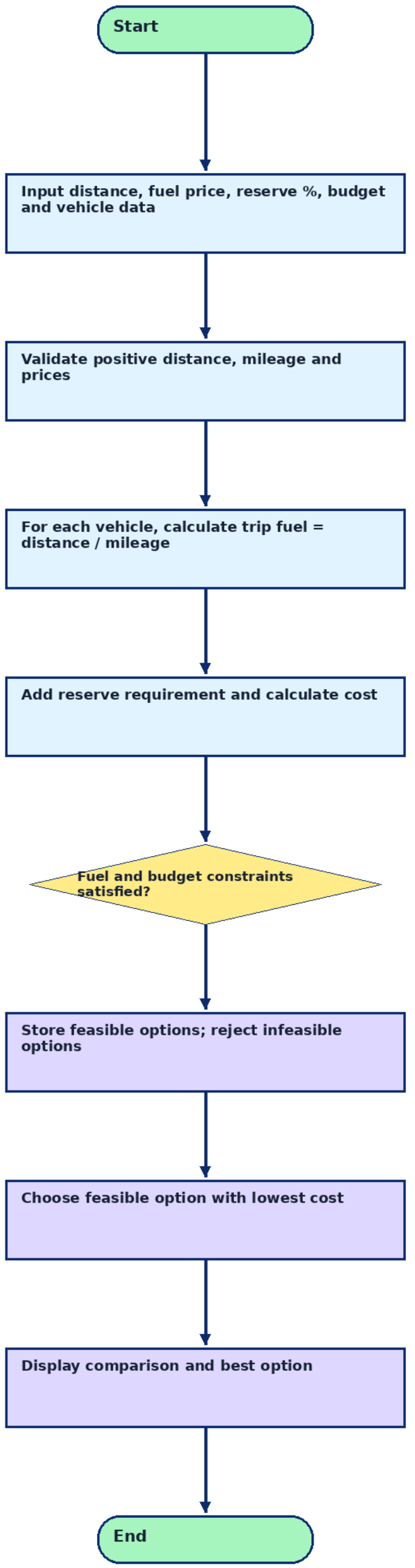
for name, v in vehicles.items():
    trip = distance_km / v["mileage"]
    needed = trip * (1 + reserve_pct / 100)
    cost = trip * fuel_price
    ok = needed <= v["fuel_available"] and cost <= budget
    print(name, f"trip={trip:.2f}L",
          f"reserve={needed:.2f}L", f"cost={cost:.2f}", ok)
    if ok:
        feasible.append((cost, name))

print("Lowest-cost feasible:",
      min(feasible)[1] if feasible else "None")
```

Sample Program Output

```
Car A trip=20.00L reserve=22.00L cost=2100.00 True
Car B trip=16.00L reserve=17.60L cost=1680.00 True
Van trip=32.00L reserve=35.20L cost=3360.00 False
Lowest-cost feasible: Car B
```

Flowchart / Procedure



9. Multi-Product Shopping Budget Constraint

Apply quantity pricing and discounts, then reduce purchases to fit the budget.

Problem Statement

Calculate each basket line using unit price and quantity, apply qualifying discounts, and compare the total with a fixed budget. If over budget, reduce quantities according to a defined priority until the basket fits; report removed units and the final total.

Python Code

```
budget = 2500
basket = [
    {"name": "Notebook", "price": 100, "qty": 8},
    {"name": "Pen pack", "price": 80, "qty": 6},
    {"name": "Bag", "price": 900, "qty": 2}
]

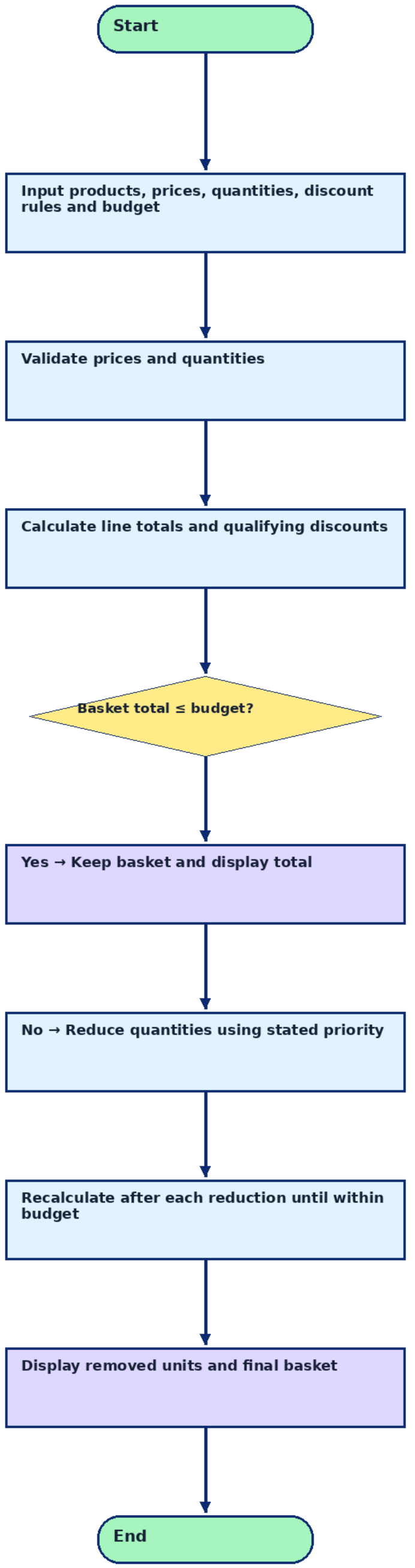
def total_cost(items):
    total = 0
    for item in items:
        line = item["price"] * item["qty"]
        discount = 0.10 * line if item["qty"] >= 6 else 0
        total += line - discount
    return total

print("Initial total:", total_cost(basket))
# Reduce lowest-priority items first: bag, pen pack, notebook
for name in ["Bag", "Pen pack", "Notebook"]:
    item = next(x for x in basket if x["name"] == name)
    while item["qty"] > 0 and total_cost(basket) > budget:
        item["qty"] -= 1
print("Final basket:", basket)
print("Final total:", total_cost(basket))
```

Sample Program Output

```
Initial total: 2760.00
Final basket:
Notebook: 8 units
Pen pack: 6 units
Bag: 1 unit
Final total: 1860.00
Within budget: Yes
```

Flowchart / Procedure



10. University Admission Decision Engine

Check course eligibility, rank qualified applicants and resolve ties consistently.

Problem Statement

Evaluate applicants using required subject marks, aggregate percentage and course-specific thresholds. Reject candidates who fail any eligibility rule. Rank eligible candidates by aggregate score, then by specified subject score and application ID as tie-breakers.

Python Code

```
courses = {
    "Computer Science": {"aggregate": 75, "math": 70},
    "Commerce": {"aggregate": 65, "math": 0}
}
applicants = [
    {"id": "A12", "course": "Computer Science",
     "aggregate": 82, "math": 78},
    {"id": "A07", "course": "Computer Science",
     "aggregate": 82, "math": 85},
    {"id": "B03", "course": "Commerce",
     "aggregate": 68, "math": 55}
]
eligible = []

for a in applicants:
    rule = courses[a["course"]]
    ok = (a["aggregate"] >= rule["aggregate"] and
          a["math"] >= rule["math"])
    print(a["id"], "Eligible" if ok else "Rejected")
    if ok:
        eligible.append(a)

eligible.sort(key=lambda a: (
    a["course"], -a["aggregate"], -a["math"], a["id"]))
print("Ranked eligible list:")
for rank, a in enumerate(eligible, start=1):
    print(rank, a["course"], a["id"], a["aggregate"], a["math"])
```

Sample Program Output

```
A12 Eligible
A07 Eligible
B03 Eligible
Ranked eligible list:
1 Computer Science A07 82 85
2 Computer Science A12 82 78
3 Commerce B03 68 55
Ranking is course-wise; subject score breaks aggregate ties.
```

Flowchart / Procedure

